Policy Iteration Optimization

2024-11-15, 14:18:27

User

print("gamma = 0.0:") policy_PI_0, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_0, epsilon=0.00001) print(policy_PI_0)

print("\ngamma = 0.1:") policy_PI_1, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_1, epsilon=0.00001) print(policy_PI_1)

print("\ngamma = 0.5:") policy_PI_5, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_5, epsilon=0.00001) print(policy_PI_5)

print("\ngamma = 0.9:") policy_PI_9, _ = infinite_horizon_policy_iteration(states, actions, transitions, rewards, gamma=gamma_9, epsilon=0.00001) print(policy_PI_9)

use for , make it more readable

2024-11-15, 14:18:28

Assistant gpt-4o-mini

Sure! You can use a loop to make the code more concise and readable. Here's how to refactor your code using a `for` loop:

```
gammas = [0.0, 0.1, 0.5, 0.9]
policies = {}

for gamma in gammas:
print(f"\ngamma = {gamma}:")
policy, _ = infinite_horizon_policy_iteration(states, actions,
transitions, rewards, gamma=gamma, epsilon=0.00001)
print(policy)
policies[gamma] = policy  # Optionally store the policies in a dictionary
for later use
```

Explanation:

1. **List of Gamma Values**: A list `gammas` is created containing the gamma values you want to use.
2. **Looping through Gamma Values**: The `for` loop iterates over each gamma value, printing the gamma value and calling the `infinite_horizon_policy_iteration` function.
3. **Storing Policies**: You can store the resulting policies in a dictionary called `policies` if you need to use them later.

This approach reduces redundancy and makes the code easier to maintain or modify.